Class 9: Delivery Pipelines, Docker, and CI/CD for ML

Data-driven Systems Engineering

Agenda

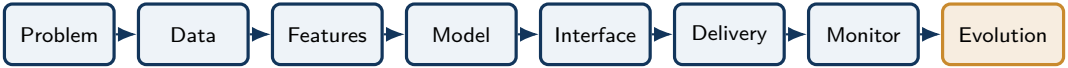
- Why delivery pipelines matter for ML systems.
- Docker, GitHub Actions, and Jenkins in the course workflow.
- Pipeline stages, gates, and release safety.
- What to test before deployment.
- Repository examples from the Flask and Streamlit apps.
- Reproducibility, release safety, and rollback thinking.

Why delivery pipelines deserve their own class

Learning goals

- Understand how packaging and automation reduce deployment friction.
- Connect notebooks and services to Docker, Jenkins, and GitHub Actions.
- Learn what should be tested before a model-backed service is released.
- Explain why CI/CD in ML must validate both software and artifacts.

Course Map and Running Cases



Today in the course

Focus: Packaging, testing, and automated delivery.
Bridge: This class moves the course from local development into repeatable release processes that reduce deployment risk.

Fraud case

In the fraud case, the scoring service becomes containerized, testable, and suitable for CI-driven release checks.

Pasteurization case

In the pasteurization case, dashboards and streaming services gain a repeatable environment so demonstrations do not depend on one laptop state.

Course thread: The key course shift is from building a system to proving that it can be rebuilt and shipped consistently.

A delivery pipeline is a risk-control mechanism

- CI/CD is not only about speed. Its main purpose is to make releases safer and more repeatable.
- In ML systems, the release unit often includes code, dependencies, configuration, and trained artifacts.
- A pipeline reduces the number of assumptions hidden inside one developer machine.

From local success to repeatable delivery

- Deterministic installs and smoke tests should happen before container release.
- Deployment is not the end; health checks and logs are part of the pipeline.

Why packaging is not optional

- Local environments drift across machines.
- Hidden dependencies can break deployments unexpectedly.
- Teams need a repeatable way to run the same code with the same artifacts.
- Packaging is part of reproducibility, not only part of operations.

Why ML delivery is harder than standard app delivery

- The build may depend on large artifacts that are not regenerated on every commit.
- A working container can still be wrong if the embedded model or scaler is stale.
- Validation must check not only whether the service starts, but whether it predicts correctly on representative inputs.
- Delivery failures may appear as silent behavior changes, not only as crashes.

Repo materials we should exploit

Repo activity

- ‘tutorials/Docker/tutorial.md‘
- ‘tutorials/Jenkins/tutorial.md‘ and ‘Jenkinsfile‘
- ‘tutorials/GitHubActions/tutorial.md‘

These are already present, so the lecture should tie them directly to the Flask and Streamlit apps.

Role of the main technologies

Docker	Builds a portable runtime for the app and its dependencies.
GitHub Actions	Runs repository-native workflows for testing, building, or deployment steps.
Jenkins	Supports more customizable automation and integration patterns.
Git	Triggers the history and events around which automation can run.

Typical stages in a small ML delivery pipeline

1. Check out code and artifacts.
2. Install dependencies in a deterministic environment.
3. Run unit or smoke tests on preprocessing and inference.
4. Build the container image.
5. Run container-level checks such as startup and health endpoints.
6. Publish or deploy only if the gates pass.

More suitable examples than generic web apps

- Build a Docker image for the fraud API and run a smoke prediction.
- Validate that the scaler artifact exists before starting the service.
- Add a GitHub Action that installs dependencies and runs a tiny inference test.
- Use Jenkins to rebuild the app when the model artifact changes.

GitHub Actions example from the tutorial

```yaml
on:
  push:
  schedule:
    - cron: "0 2 * * *"

jobs:
  test:
    runs-on: ubuntu-latest
```

- ‘tutorials/GitHubActions/tutorial.md‘ is useful because it shows that workflows are event-driven.
- For this course, the key idea is that pushes and scheduled checks can both guard the service.

Jenkins example from the repository

```
stage('Build') {
  steps {
    sh 'docker build -t $IMAGE:$BUILD_NUMBER .'
  }
}
stage('Test') {
  steps {
    sh 'docker run --rm $IMAGE:$BUILD_NUMBER npm test'
  }
}
```

- ‘tutorials/Jenkins/Jenkinsfile‘ makes the pipeline stages explicit.
- The exact commands may change for Python services, but the structure of build, test, and release gates remains the same.

What to test in ML delivery

- Schema validation for inputs and outputs.
- Serialization compatibility for model artifacts.
- Inference latency under small load.
- Reproducibility of a baseline metric on a frozen sample.
- Container startup and health endpoint behavior.

Release gates for model-backed services

- Does the artifact load with the current runtime and library versions?
- Does a frozen inference sample still produce acceptable output?
- Does the container expose the required route and health response?
- Are secrets, environment variables, and external dependencies configured correctly?

What can go wrong without CI/CD

- A model artifact is missing or incompatible with the code.
- The container starts but the inference route fails.
- A dependency update changes prediction behavior silently.
- A dashboard works locally but not in the release environment.

Rollback and observability are part of delivery

- Safe delivery includes a way to revert a bad release quickly.
- Observability tells us whether a release is healthy after deployment.
- Logs, metrics, and health endpoints are part of the delivery design, not a postscript.

Course connection

This is the bridge from software engineering into operational MLOps: the deployment pipeline and the monitoring pipeline eventually meet.

Technology links

Papers and materials

Docker get started; GitHub Actions documentation; Jenkins documentation; Docker CLI reference.

From This Class to the Next

What stays with us

- Delivery is a repeatable engineering process, not a one-time handoff.
- Containers and CI/CD reduce environment drift and manual release mistakes.
- Release quality depends on tests, health checks, rollback paths, and observability.

Project use

Teams can now describe how code, dependencies, models, and checks travel from the repository into a running service.

Next connection

Class 10 steps back to requirements engineering so these technical pipelines are anchored to explicit stakeholder needs and acceptance criteria.

Course thread: Reliable delivery matters most when the team knows exactly what the system is expected to do and how success will be judged.

Papers and materials

Papers and materials

Docker documentation; GitHub Actions documentation; Jenkins documentation; Continuous Delivery for Machine Learning.